

# Distilled Prompt Tuning for Efficient GNN-based Recommendation Unlearning

Anonymous Author(s)

## Abstract

In GNN-based recommender systems, all users and items share global model parameters. Thus, existing unlearning methods typically have to modify these shared parameters to unlearn specific user-item interactions, which may cause either costly retraining or unintentional degradation of recommendation quality for other users. To avoid this global side effect, we propose the Distillation Prompt Unlearning (DPU) framework. Instead of modifying the entire model, DPU freezes the backbone and distills knowledge into lightweight prompt vectors, enabling targeted preservation of retained data and selective removal of forgotten interactions. Specifically, a “*Memorizing Teacher*” serves as a stable reference of the original model behavior, guiding the prompt to preserve reliable ranking patterns for unaffected nodes. In contrast, the “*Forgetting Teacher*” utilizes an unlearned reference to disrupt the signals of target interactions, making them indistinguishable from random noise. We further combine these distillation signals into a unified optimization objective and provide theoretical guarantees on the ability of prompt vectors to simulate structural unlearning. Extensive experiments on three benchmark datasets demonstrate that DPU significantly outperforms state-of-the-art baselines in unlearning effectiveness, model utility, and computational efficiency. The source code is available at: <https://anonymous.4open.science/r/Prompt-Unlearning>

## CCS Concepts

• Information systems → Recommender systems.

## Keywords

Recommendation Unlearning; Prompt Tuning; Graph Neural Networks; Knowledge Distillation; Efficient Unlearning

## ACM Reference Format:

Anonymous Author(s). 2026. Distilled Prompt Tuning for Efficient GNN-based Recommendation Unlearning. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 10 pages. <https://doi.org/XXXXXX.XXXXXXX>

## 1 Introduction

Recommender systems are the information filtering backbone of the modern web, translating user-item interactions into personalized experiences [12, 13]. This reliance on historical data creates

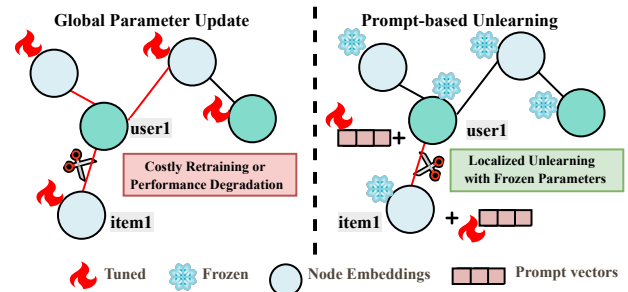
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

Conference acronym 'XX, Woodstock, NY

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X /2018/06

<https://doi.org/XXXXXX.XXXXXXX>



**Figure 1: Prompt-based unlearning versus parameter-updating unlearning.** DPU achieves targeted unlearning by injecting lightweight prompts into frozen node embeddings, enabling selective removal of interaction influence without modifying shared model parameters.

challenges. Beyond the legal requirements of the “right to be forgotten” (e.g., GDPR [29]), the practical necessity for Recommendation Unlearning (RU) arises from the dynamic nature of user intent. A user may inadvertently click on “clickbait” and become trapped in a filter bubble of toxic content, or undergo life transitions (e.g., changing jobs) where outdated suggestions become irrelevant. Failing to erase these interactions timely violates privacy and degrades recommendation quality [2, 14]. Thus, *recommendation unlearning* seeks to eliminate the influence of selected data from a trained model while preserving utility for unaffected users and interactions.

Unlearning is particularly challenging in Graph Neural Network (GNN) based recommenders. GNNs learn representations by recursively propagating and transforming information across the user-item topology [36, 41]. Crucially, this process relies on a set of *global, shared* convolutional operators to aggregate information for *every* user/items. This architecture creates a fundamental conflict: while an unlearning request is *local* (deleting an edge), the related information is embedded into the *global* parameters. Consequently, any modification for specific user-item unlearning needs either costly retraining or leads to a degradation in overall recommendation quality. Existing unlearning methods struggle to effectively address this trade-off in GNN architectures. Specifically, (i) Partition-based methods, such as SISA [1], RecEraser [2], and LASER [23], retrain affected data shards, but this fragments the global graph structure and harms recommendation quality. (ii) Fine-tuning-based methods, including Unlearn-ALS [35] and AltEraser [23], are efficient but rely on heuristic parameter adjustments that often result in incomplete forgetting. (iii) Influence-function-based methods, such as IFRU [42], SCIF [20], SeqIF [37], and SessIF [34], utilize linear Hessian approximations to estimate data influence, but can be computationally intensive and prone to approximation errors.

To overcome these limitations, we introduce a novel approach that reframes GNN-based recommendation unlearning as a lightweight adaptation task by distilling tailored prompt vectors for

individual nodes. Inspired by recent advances in graph prompting [9, 28, 36], we design “memorizing” prompt vectors for nodes to retain information and “forgotten” prompt vectors for interactions to be unlearned. However, applying prompting to GNN unlearning presents two primary difficulties: (1) *Efficiency and scalability*: Assigning a distinct prompt vector to every node is memory and computation intensive [21]. This hinders scalability on large-scale recommendation graphs with millions of users and items. (2) *Effective and targeted forgetting*: Due to the interconnected nature of graphs, deleting a single edge perturbs high-order neighborhoods [4, 21]. How to design prompts that make a model forget specific interactions without hurting recommendation quality is still challenging.

To address these challenges, we propose a **Distillation Prompt Unlearning (DPU)** framework, the first framework reframes unlearning as a distilled prompt-tuning task. As illustrated in the right part of Figure 1, DPU freezes the pre-trained GNN and tunes a small set of learnable prompt vectors injected into the node embeddings. These prompts simulate the structural effect of edge deletion within the embedding space, avoiding costly model updates. To address the first challenge, we employ an attention-based prompt construction. This method generates node prompts using a compact set of shared basis vectors, reducing parameters and enabling efficient tuning. To guide effective forgetting, we employ a dual-teacher knowledge distillation strategy. A memorizing teacher pre-trained on the full dataset preserves high-quality ranking patterns to support utility preservation, while a randomly initialized forgetting teacher, unexposed to the target interactions, drives the removal of designated information. Since standard distillation is ineffective with discrete ranking and unlearning, we introduce three specialized objectives: *Structure-Aware Forgetting Distillation (SFD)* guides the prompt to modify node embeddings to weaken or sever specific graph links and nullify interaction influence. *Position-Aware Ranking Distillation (PRD)* and *Global Kendall Regularization (GKR)* jointly preserve ranking quality and global stability for unaffected users. The main contributions of this work are summarized as follows:

- We propose DPU, the first **prompt-based** framework for recommendation unlearning. It freezes the GNN backbone and achieves unlearning by tuning lightweight prompts in the node embedding space, enabling reversible unlearning without retraining.
- We propose **Topology-aware prompt tuning** for GNN recommenders, showing that prompts can simulate the structural effect of edge deletion by reshaping latent message-passing outcomes rather than updating model parameters.
- We develop a **dual-teacher distillation** scheme tailored to implicit-feedback ranking, with three objectives (PRD, SFD, GKR) that jointly balance forgetting completeness, top- $K$  utility preservation, and global ranking stability.
- We provide theoretical analysis supporting prompt-based unlearning and validate DPU on three benchmarks, achieving consistent performance gains over state-of-the-art baselines in unlearning effectiveness, model utility, and computational efficiency.

## 2 Related Work

**Recommendation Unlearning.** Recommendation unlearning seeks to remove the influence of specific users, items, or interactions

from trained models for privacy compliance or continual updates. Existing methods broadly fall into three categories. (1) *Partition-based methods*, such as RecEraser [2] and LASER [23], follow the SISA paradigm by partitioning training data and retraining only affected shards. UltraRE [18] further improves this line of work via error decomposition to better balance utility and efficiency. However, these approaches physically fragment the user-item graph and can weaken global collaborative signals. (2) *Fine-tuning-based methods*, including Unlearn-ALS [35] and AltEraser [23], directly update model parameters and often struggle to achieve complete forgetting without degrading retained knowledge. (3) *Influence-function-based methods* use influence functions [34, 37, 42] or structural operators such as GNNDelete [5] to estimate the impact of removed data. Despite their efficiency, influence-based methods typically rely on linear Hessian approximations, which are poorly matched to GNN recommenders where edge deletions induce non-linear, cascading changes across the graph.

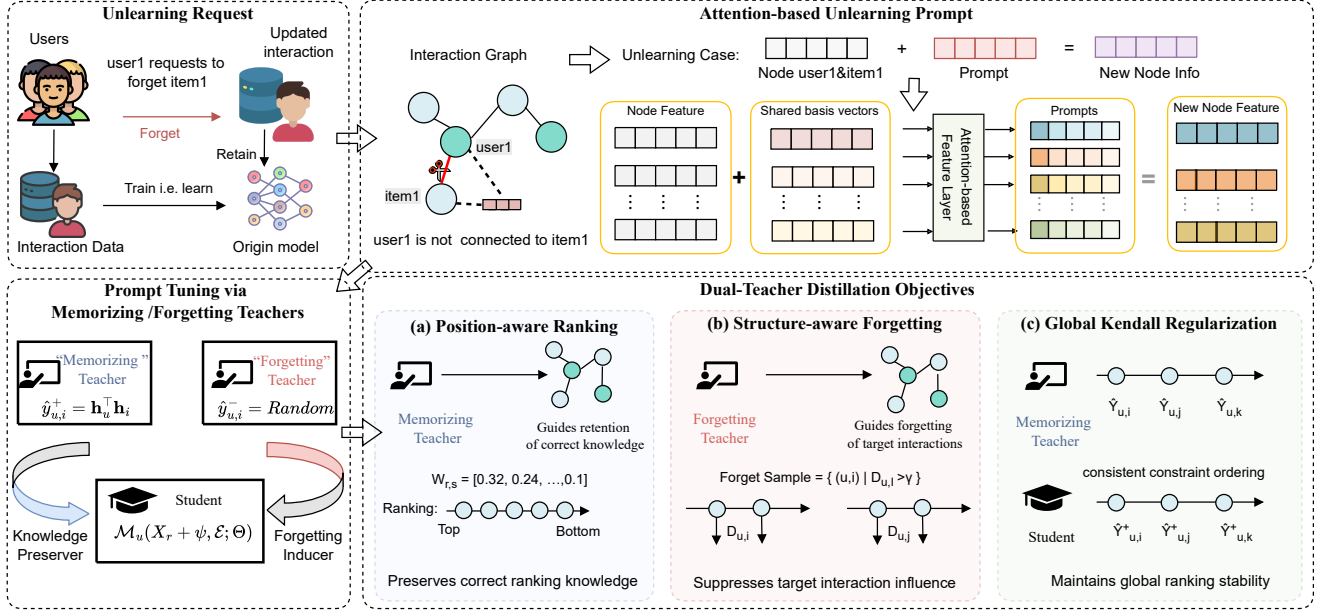
**Prompt-based Learning for GNNs.** Prompt-based learning, originating from NLP [8, 15, 39], has emerged as a powerful paradigm for efficiently adapting large pre-trained models. This technique is successfully extended to GNNs and recommender systems [22, 24], where lightweight, learnable prompts are tuned to guide the model’s behavior for new tasks while keeping the backbone frozen. For example, GraphPro [36] employs temporal and structural prompts to adapt GNNs to dynamic user behaviors, while GPT4Rec [40] introduces multi-view graph prompts at node, structure, and view levels to enhance streaming recommendations. Similarly, the GPF framework [9] uses task-agnostic prompt tuning over graph features to adapt pre-trained GNNs efficiently. However, existing prompt frameworks are designed for forward adaptation—injecting new knowledge or aligning features. They lack mechanisms for the targeted negation and structural erasure required for unlearning.

**Knowledge Distillation for Unlearning.** Knowledge distillation has also been explored as an efficient mechanism for unlearning [17, 30, 31]. Early graph-based distillation methods focus on utility preservation by transferring knowledge from a teacher trained on retained data, typically using a single-teacher setting [43]. To explicitly enforce forgetting, recent works adopt dual-teacher distillation strategies, where different teachers are used to separately guide retention and removal of information [6, 27, 44]. These methods employ specialized teacher roles or masking mechanisms to disentangle forgetting and preservation signals on graphs. Nevertheless, they generally require modifying core model parameters. How to leverage dual-teacher distillation to guide *prompt tuning*, particularly to simulate discrete structural deletions in graph recommenders, remains an open problem that we address in this work.

## 3 Preliminaries and Problem Formulation

We first introduce notation and formalize prompt-based recommendation unlearning under implicit feedback with a bipartite user-item graph.

*Definition 3.1 (User-Item Interaction Graph).* Let  $\mathcal{U} = \{u_1, \dots, u_m\}$  and  $\mathcal{I} = \{i_1, \dots, i_n\}$  denote the sets of  $m$  users and  $n$  items, respectively. We define the user-item interaction graph as a bipartite graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ , where  $\mathcal{V} = \mathcal{U} \cup \mathcal{I}$  is the set of all nodes ( $|\mathcal{V}| = m + n$ ), and  $\mathcal{E}$  is the set of edges representing observed interactions. The



**Figure 2: The framework of DPU. Step 1: The user requests unlearning. Step 2: A prompt is designed and added to the input features to remove target knowledge. Step 3: The prompt is tuned using two teachers and three distillation strategies to forget the target while preserving model utility.**

initial node representations are given by an embedding matrix  $X \in \mathbb{R}^{|\mathcal{V}| \times d}$ , where  $d$  is the latent dimension. In ID-based recommenders (e.g., LightGCN),  $X$  corresponds to the learnable 0-th layer ID embeddings  $E^{(0)}$ .

**Definition 3.2 (GNN-based Recommender).** A GNN-based recommender, denoted as  $\mathcal{M}(X, \mathcal{E}; \Theta)$ , operates on the graph structure to refine node embeddings. Given the initial embeddings  $X$  and edge set  $\mathcal{E}$ , the model performs message passing to compute the final representations  $H \in \mathbb{R}^{|\mathcal{V}| \times d}$ . The prediction for a user-item pair  $(u, i)$  is computed as  $\hat{y}_{u,i} = h_u^\top h_i$ . The parameters  $\Theta$  are typically pre-trained on the complete graph  $\mathcal{G}$  using a ranking objective, such as the Bayesian Personalized Ranking (BPR) loss:

$$\mathcal{L}_{\text{BPR}} = \sum_{(u,i^+,i^-) \in \mathcal{D}} -\ln \sigma(\hat{y}_{u,i^+} - \hat{y}_{u,i^-}) + \lambda \|\Theta\|_2^2, \quad (1)$$

where  $\mathcal{D}$  represents the training triples constructed from  $\mathcal{E}$ .

**Definition 3.3 (Forget and Retain Sets).** For interaction-level unlearning, the edge set  $\mathcal{E}$  is partitioned into two disjoint subsets: a forget set  $\mathcal{E}_f$  and a retain set  $\mathcal{E}_r$ , such that  $\mathcal{E}_f \cup \mathcal{E}_r = \mathcal{E}$  and  $\mathcal{E}_f \cap \mathcal{E}_r = \emptyset$ . The corresponding subgraphs are denoted as  $\mathcal{G}_f = (\mathcal{V}, \mathcal{E}_f)$  and  $\mathcal{G}_r = (\mathcal{V}, \mathcal{E}_r)$ . Ideally, the unlearned model should behave as if it were trained solely on the retain subgraph  $\mathcal{G}_r = (\mathcal{V}, \mathcal{E}_r)$ . The node embedding matrices extracted from  $\mathcal{G}_r$  and  $\mathcal{G}_f$  are denoted as  $X_r$  and  $X_f$ , respectively.

**Definition 3.4 (Prompt Matrix).** To avoid expensive retraining, we introduce a continuous prompt matrix  $\psi \in \mathbb{R}^{|\mathcal{V}| \times d}$ . Unlike discrete structural pruning,  $\psi$  acts as an additive perturbation to the global node embeddings  $X$ . The prompted input becomes  $\tilde{X} = X + \psi$ . This continuous modulation aims to simulate the spectral and structural effects of removing  $\mathcal{E}_f$  within the embedding space, without altering the frozen backbone parameters  $\Theta$ .

**Problem Statement.** Given a pre-trained GNN-based recommender  $\mathcal{M}_{\text{pre}}(X, \mathcal{E}; \Theta)$  trained on the full interaction graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ , a forget set  $\mathcal{E}_f$ , and a retain set  $\mathcal{E}_r$ , the objective is to learn a prompt matrix  $\psi \in \mathbb{R}^{|\mathcal{V}| \times d}$  such that the prompted model  $\mathcal{M}_u(X_r + \psi, \mathcal{E}; \Theta)$  satisfies three criteria: (ii) *Unlearning Efficiency*: computational cost is minimized compared to full retraining; and (iii) *Model Utility*: performance on  $\mathcal{E}_r$  is preserved, ideally matching a model trained exclusively on  $\mathcal{E}_r$ .

## 4 Methodology

We propose DPU, a lightweight unlearning framework that reformulates recommendation unlearning as a prompt-guided adaptation task rather than parameter optimization. As illustrated in Figure 2, DPU comprises two key components. We begin by introducing the Attention-based Unlearning Prompt (Section 4.1), the core mechanism that injects lightweight, reversible changes into the input feature space. This design generates node-specific prompts from a compact set of shared basis vectors, ensuring efficiency and scalability. Next, we detail the Prompt Distillation via Dual-Teacher strategy (Section 4.2), which guides the optimization of these prompts with the tailored distillation objectives.

### 4.1 Attention-based Unlearning Prompt

To enable efficient and reversible unlearning without changing the model architecture or retraining, we inject a learnable prompt matrix  $\psi \in \mathbb{R}^{|\mathcal{V}| \times d}$  into the input node feature matrix  $X$ , forming the final input  $\tilde{X} = X + \psi$  to the frozen GNN recommender  $\mathcal{M}_{\text{pre}}$ .

However, a naive implementation of this idea, which assigns a unique and independent learnable prompt vector to every node is inefficient and unscalable for large graphs. For large-scale recommender systems with millions of users and items, the memory

and computational cost of learning and storing a  $|\mathcal{V}| \times d$  prompt matrix becomes prohibitive. To overcome this bottleneck, we propose an *attention-based prompt constructor* that generates node-specific prompts via a dynamic composition mechanism. Specifically, we instantiate the prompt generator with two compact sets of learnable parameters: (1) a shared pool of basis prompt vectors  $\mathcal{B} = \{\psi_1^b, \dots, \psi_{K_p}^b\}$ , and (2) a corresponding set of attention query vectors  $\mathcal{Q} = \{\mathbf{a}_1, \dots, \mathbf{a}_{K_p}\}$ , where  $K_p \ll |\mathcal{V}|$  and  $\mathbf{a}_k \in \mathbb{R}^d$ .

For any given node  $v$ , the specific prompt  $\psi_v$  is synthesized as a weighted combination of the basis vectors in  $\mathcal{B}$ , where the weights are determined by matching the node's frozen embedding  $\mathbf{h}_v$  against the query vectors in  $\mathcal{Q}$ . Formally, the generation process is defined as:

$$\psi_v = \sum_{k=1}^{K_p} \alpha_{v,k} \cdot \psi_k^b, \quad \alpha_{v,k} = \frac{\exp(\mathbf{a}_k^\top \mathbf{h}_v)}{\sum_{j=1}^{K_p} \exp(\mathbf{a}_j^\top \mathbf{h}_v)}, \quad (2)$$

where  $\mathbf{h}_v \in \mathbb{R}^d$  corresponds to the frozen initial embedding of node  $v$  (i.e., the  $v$ -th row of  $\mathbf{X}$ ). This attention-based construction substantially reduces the number of prompt parameters, generalizes to unseen nodes, and supports efficient, plug-and-play unlearning.

## 4.2 Prompt Distillation via Dual-Teacher

Inspired by dual-teacher unlearning [6], we develop a prompt-tuning framework designed for recommendation unlearning. The memorizing teacher  $\mathcal{T}^+$  (frozen copy of the original model trained on  $\mathcal{G}_r$ ) guides the optimization of memorizing prompts to retain accurate ranking knowledge. Simultaneously, the forgetting teacher  $\mathcal{T}^-$  (randomly initialized) guides the forgetting prompts to disrupt target interaction signals, making them indistinguishable from random noise. However, standard KL-based distillation is not suitable for implicit-feedback ranking for two reasons: (1) it matches the full output distribution, but recommendation is a top- $K$  ranking task, not a full distribution problem; and (2) it treats unobserved interactions as negatives, which is inaccurate for implicit feedback and can interfere with unlearning.

To address these issues, we introduce three complementary objectives: (i) **Position-Aware Ranking Distillation**, which uses the memorizing teacher to emphasize top-ranked items; (ii) **Structure-Aware Forgetting Distillation**, which leverages the forgetting teacher to suppress target interactions; and (iii) **Global Kendall Regularization**, which aligns with the memorizing teacher to preserve global ranking consistency. A shared hyperparameter  $K_r$  defines the preservation scope. By supervising different prompt sets on disjoint interaction subsets, the teachers provide complementary rather than conflicting signals.

**4.2.1 Position-Aware Ranking Distillation.** To preserve recommendation performance in the retained set  $\mathcal{G}_r$ , we adopt Position-Aware Ranking Distillation (PRD), inspired by *Ranking Distillation* [26]. PRD concentrates on the top- $K$  items ranked by the memorizing teacher  $\mathcal{T}^+$ , reflecting the practical emphasis on top- $N$  recommendation quality. By explicitly preserving the relative order among highly ranked items, PRD helps the student prompt maintain high-quality recommendations.

For a user  $u \in \mathcal{U}_r$ , let the teacher's top- $K_r$  ranking be denoted by  $\pi_u^T = \{i_{u,1}^T, i_{u,2}^T, \dots, i_{u,K_r}^T\}$ , where items are sorted by teacher

scores  $\hat{y}_{ui}^+ = \mathcal{T}^+(u, i)$  in descending order. The student model  $\mathcal{M}_u$  predicts scores  $\hat{y}_{ui}$  from prompt-enhanced features  $\tilde{\mathbf{h}}_v$  as  $\hat{y}_{ui} = \mathcal{M}_u(\tilde{\mathbf{h}}_v, \mathcal{E})[u, i]$ . For each item pair  $(i_{u,r}^T, i_{u,s}^T)$  with  $r < s$ , we enforce a pairwise constraint that the student preserves the teacher's preference order. To emphasize accuracy at higher ranks, we assign a decaying weight to each item pair based on the position  $r$  of the higher-ranked item in the teacher's list. Specifically, the weight is defined as:

$$w_{r,s} = \exp(-\alpha_w \cdot r), \quad (3)$$

where a smaller  $r$  (closer to the top) gives a larger weight, and  $\alpha_w$  controls how fast the weight decays. The distillation loss encourages the student to match the teacher's top- $K$  ranking. For each user  $u$ , and for every item pair  $(i_{u,r}^T, i_{u,s}^T)$  in the teacher's ranking  $\pi_u^T$  where  $r < s$ , we enforce:

$$\mathcal{L}_{\text{PRD}} = - \sum_{u \in \mathcal{U}_{\text{batch}}} \sum_{1 \leq r < s \leq K_r} w_{r,s} \cdot \log \sigma(\hat{y}_{u,i_{u,r}^T} - \hat{y}_{u,i_{u,s}^T}), \quad (4)$$

where  $\hat{y}_{u,i}$  is the student's predicted score for user  $u$  and item  $i$ , and  $\sigma(\cdot)$  is the sigmoid function. The term inside the log encourages the student to rank  $i_{u,r}^T$  higher than  $i_{u,s}^T$ , aligning with the teacher's preference.

As the student is initialized from the fully trained  $\mathcal{T}^+$  on  $\mathcal{G}$ , we employ a stochastic optimization strategy by sampling a mini-batch of users  $\mathcal{U}_{\text{batch}} \subset \mathcal{U}_r$  at each iteration for distillation. This avoids computationally expensive full-graph re-ranking while effectively preserving the key ranking quality.

**4.2.2 Structure-Aware Forgetting Distillation.** To selectively unlearn user-item interactions in the forget set  $\mathcal{G}_f$ , we guide the student  $\mathcal{M}_u$  to align with the forgetting teacher  $\mathcal{T}^-$  on forget-critical samples. Because  $\mathcal{T}^-$  is not trained on  $\mathcal{G}_f$ , its predictions contain no influence from the forget set, making it a natural guide for forgetting. However, for many  $(u, i) \in \mathcal{G}_f$ ,  $\mathcal{M}_u$  and  $\mathcal{T}^-$  already produce similar scores; distilling on these low-risk cases adds noise and can degrade performance. To identify forget-critical samples, we compute the prediction gap:

$$\Delta_{u,i} = |\hat{y}_{u,i} - \mathcal{T}^-(u, i)|, \quad (5)$$

and select samples with significant discrepancy:

$$\mathcal{S}_f = \{(u, i) \in \mathcal{E}_f \mid \Delta_{u,i} > \gamma_t\}, \quad (6)$$

where  $\gamma_t$  is a predefined threshold. For each user  $u$ , we construct a candidate set of  $K_r$  items: one target  $(u, i) \in \mathcal{S}_f$  and  $K_r - 1$  random items that user  $u$  has not interacted with in the forget set. Over these  $K$  items, we apply the softened KL divergence to align the student's logits  $\hat{y}_{u,1:K}$  with the forgetting teacher's logits  $\hat{y}_{u,1:K}^-$ :

$$\mathcal{L}_{\text{SFD-KL}} = \sum_{u \in \mathcal{U}_f} \text{KL} \left( \text{softmax} \left( \frac{\hat{y}_{u,1:K_r}}{\tau} \right) \parallel \text{softmax} \left( \frac{\hat{y}_{u,1:K_r}^-}{\tau} \right) \right), \quad (7)$$

where  $\tau$  is a temperature hyperparameter controlling the smoothness of the score distribution. The final loss is:

$$\mathcal{L}_{\text{SFD}} = \lambda_1 \cdot \mathcal{L}_{\text{SFD-KL}} + \lambda_2 \cdot \sum_{(u,i) \in \mathcal{S}_f} \sum_{j \in \mathcal{I}_u^-} \max(0, \hat{y}_{u,i} - \hat{y}_{u,j} + \delta), \quad (8)$$

where the second term introduces a contrastive margin loss to ensure that forget-set items  $(u, i)$  receive lower scores than unrelated negative items  $(u, j)$  by at least a margin  $\delta$ , preventing the student from simply assigning low scores to all items without distinction.  $\lambda_1$  and  $\lambda_2$  are hyperparameters that balance the contributions of the KL alignment and contrastive components.

**4.2.3 Global Kendall Regularization.** While the previous distillation objectives emphasize item-level relevance, we introduce *Global Kendall Regularization* (GKR) to preserve the *holistic ranking structure* learned by the memorizing teacher. GKR protects the student from drifting in its overall item ordering, a common failure mode when unlearning large or noisy forget sets. By penalizing pairwise ordering disagreements, GKR acts as a structural constraint that anchors the student’s global ranking policy, ensuring that unlearning does not disturb relative item orderings beyond the targeted removals.

For each user  $u \in \mathcal{U}$ , we construct  $C_u$  by selecting the top- $K_r$  items ranked by the memorizing teacher  $\mathcal{T}^+$  over the retained graph  $\mathcal{G}_r$ , including both interacted and unobserved items. Let  $\hat{y}_{ui}$  and  $\hat{y}_{ui}^+$  denote the student and teacher predictions, respectively. Inspired by [16], we define the GKR loss as a smooth approximation of the Kendall Tau distance between the teacher and student rankings over  $C_u$ .

$$\mathcal{L}_{\text{GKR}} = \sum_{u \in \mathcal{U}} \frac{2}{K_r(K_r - 1)} \sum_{\substack{i, j \in C_u \\ i < j}} \sigma\left(-(\hat{y}_{u,i} - \hat{y}_{u,j})(\hat{y}_{u,i}^+ - \hat{y}_{u,j}^+)\right), \quad (9)$$

where  $\sigma(\cdot)$  is the sigmoid function approximating the indicator of pairwise ranking inconsistency. The normalization term  $\frac{2}{K_r(K_r - 1)}$  ensures that the loss remains within  $[0, 1]$  for each user. This expression penalizes item pairs whose relative order is reversed by the student compared to the teacher.

### 4.3 Training Objective

We freeze all backbone parameters  $\bar{\Theta}$  inherited from the memorizing teacher  $\mathcal{T}^+$  and optimize only the prompt parameters  $\psi$  for unlearning. The training objective combines the standard recommendation loss with three unlearning-oriented distillation losses:

$$\min_{\psi} \underbrace{\mathcal{L}_{\text{BPR}}}_{\text{Rec. loss}} + \underbrace{\alpha \mathcal{L}_{\text{PRD}} + \beta \mathcal{L}_{\text{SFD}} + \gamma \mathcal{L}_{\text{GKR}}}_{\text{Distillation loss}}. \quad (10)$$

The first part of the objective,  $\mathcal{L}_{\text{BPR}}$  is the standard recommendation loss that equips the model with basic ranking capability on the retained data. The second part is the distillation loss for unlearning, which integrates three components:  $\mathcal{L}_{\text{PRD}}$  for preserving ranking quality,  $\mathcal{L}_{\text{SFD}}$  for enforcing forgetting on sensitive interactions, and  $\mathcal{L}_{\text{GKR}}$  for maintaining global ranking consistency. The weights  $\alpha, \beta$ , and  $\gamma$  balance the impact of each component. To provide a holistic view of the DPU framework, we present the complete optimization procedure in Algorithm 1.

### 4.4 Theoretical Analysis

To justify the effectiveness of DPU, we provide theoretical guarantees on prompt-driven structural approximation.

**4.4.1 Theoretical Analysis on Structural Approximation.** We first establish the existence of an optimal prompt (Theorem 4.1) on the ability of prompt vectors to simulate structural unlearning.

**THEOREM 4.1 (APPROXIMATION CAPABILITY OF PROMPTS FOR STRUCTURAL UNLEARNING).** *Let  $f(\mathbf{A}, \mathbf{X}) = \sigma(\tilde{\mathbf{A}}\mathbf{X}\mathbf{W})$  be a Graph Convolutional Network (GCN) layer, where  $\tilde{\mathbf{A}}$  is the normalized adjacency matrix and  $\mathbf{W} \in \mathbb{R}^{d \times d'}$  is the learnable weight matrix. Let  $\mathcal{G} = (\mathbf{A}, \mathbf{X})$  be the original graph and  $\mathcal{G}' = (\mathbf{A}', \mathbf{X})$  be the unlearned graph where  $\mathbf{A}' = \mathbf{A} - \Delta\mathbf{A}$  corresponds to deleting edges in  $\mathcal{E}_f$ . Even if  $\tilde{\mathbf{A}}$  is singular and  $\mathbf{W}$  is not full-rank, there exists an optimal prompt  $\psi^*$  that minimizes the Frobenius norm of the pre-activation unlearning error,  $\epsilon = \|\tilde{\mathbf{A}}(\mathbf{X} + \psi)\mathbf{W} - \tilde{\mathbf{A}}'\mathbf{X}\mathbf{W}\|_F$ .*

**PROOF.** To simulate unlearning, we seek a prompt  $\psi$  such that the pre-activation output  $f(\mathbf{A}, \mathbf{X} + \psi)$  approximates the retrained model’s,  $f(\mathbf{A}', \mathbf{X})$ . This yields the linear matrix equation for  $\psi$ :

$$\tilde{\mathbf{A}}(\mathbf{X} + \psi)\mathbf{W} = \tilde{\mathbf{A}}'\mathbf{X}\mathbf{W}. \quad (11)$$

Let  $\Delta\tilde{\mathbf{A}} = \tilde{\mathbf{A}}' - \tilde{\mathbf{A}}$  be the change in the normalized adjacency matrix. Rearranging Eq. (11) yields a classic Sylvester-like equation:

$$\tilde{\mathbf{A}}\psi\mathbf{W} = \Delta\tilde{\mathbf{A}}\mathbf{X}\mathbf{W}. \quad (12)$$

In practical recommendation scenarios, exact solutions to Eq. (12) may not exist. This is because  $\tilde{\mathbf{A}}$  is often singular (due to sparse or disconnected graph components) and  $\mathbf{W}$  is typically rank-deficient ( $d' < d$ ). Consequently, the target perturbation  $\mathbf{T} = \Delta\tilde{\mathbf{A}}\mathbf{X}\mathbf{W}$  may lie outside the column space spanned by the original graph operator  $\tilde{\mathbf{A}}(\cdot)\mathbf{W}$ . However, the problem can be formulated as finding a Least Squares solution that minimizes the approximation error  $\|\tilde{\mathbf{A}}\psi\mathbf{W} - \mathbf{T}\|_F$ . According to generalized inverse theory [25], the optimal solution with minimum Frobenius norm is analytically given by:

$$\psi^* = \tilde{\mathbf{A}}^\dagger (\Delta\tilde{\mathbf{A}}\mathbf{X}\mathbf{W})\mathbf{W}^\dagger, \quad (13)$$

where  $(\cdot)^\dagger$  denotes the Moore-Penrose pseudoinverse. The existence of  $\psi^*$  is guaranteed for any matrix  $\tilde{\mathbf{A}}$  and  $\mathbf{W}$ . This confirms a feature-space prompt can simulate edge deletion. While DPU does not compute  $\psi^*$  analytically, its distillation-guided optimization numerically finds a prompt that approximates this optimal, minimum-error solution, validating our approach.  $\square$

**4.4.2 Complexity Analysis.** We analyze the efficiency of DPU compared to full retraining. Let  $L$  be the number of layers,  $|\mathcal{E}|$  the number of edges, and  $d$  the embedding dimension.

- **Time Complexity:** Full retraining has a complexity of  $O(L \cdot |\mathcal{E}| \cdot d)$  per epoch. DPU only updates prompt parameters, with gradients computed on the retain set, leading to a complexity of  $O(L \cdot |\mathcal{E}_r| \cdot d)$  per epoch. As we only tune a small parameter set, DPU converges in significantly fewer epochs.
- **Space Complexity:** DPU introduces a small overhead of  $O(K_p \cdot d + |\mathcal{V}| \cdot K_p)$  for the prompt basis vectors and attention weights. Given  $K_p \ll |\mathcal{V}|$ , this is negligible compared to storing full model parameters.

## 5 Experiments

In this section, we conduct extensive experiments to evaluate the performance of our proposed DPU framework. Our evaluation aims to address the following four research questions:

## Algorithm 1: Distillation Prompt Unlearning (DPU) Algorithm

**Input:**  $\mathcal{G}$ : Original graph with adjacency matrix  $\mathbf{A}$  and features  $\mathbf{X}$ ;  $\mathcal{E}_f, \mathcal{E}_r$ : Forget and Retain interaction sets;  $\mathcal{M}_{\text{pre}}$ : Pretrained GNN backbone;  $\mathcal{P}$ : Prompt generator.

**Output:** Recommendation scores  $\hat{y}_u$  for user  $u$ .

```

1: Initialization:
2:  $\tilde{\Theta} \leftarrow \mathcal{M}_{\text{pre}}(\mathbf{X}, \mathcal{E})$  {Load frozen backbone parameters}
3:  $\psi \leftarrow \mathcal{P}(\mathbf{X}_r)$  {Initialize prompt for retained set  $\mathcal{E}_r$ }
4:  $\mathcal{T}^+ \leftarrow \mathcal{M}_{\text{pre}}(\mathbf{X}, \mathcal{E})$  {Initialize Memorizing Teacher}
5:  $\mathcal{T}^- \leftarrow \text{RandomInit}(\mathcal{G}_f)$  {Initialize Forgetting Teacher}
6: Unlearning Optimization:
7: while not converged do
8:    $\tilde{\mathbf{X}} = \mathbf{X}_r + \psi$  {Inject prompt into features}
9:    $\hat{y} = \mathcal{M}(\tilde{\mathbf{X}}, \mathcal{E}_r; \tilde{\Theta})$  {Student forward pass}
10:  Teacher Guidance:
11:   Get  $\hat{y}^+ = \mathcal{T}^+(u, i)$  for  $(u, i) \in \mathcal{E}_r$ 
12:   Get  $\hat{y}^- = \mathcal{T}^-(u, i)$  for  $(u, i) \in \mathcal{E}_f$ 
13:  Compute Losses:
14:   Calculate  $\mathcal{L}_{\text{PRD}}$  using Memorizing Teacher  $\hat{y}^+$  (Eq. 4)
15:   Calculate  $\mathcal{L}_{\text{SFD}}$  using Forgetting Teacher  $\hat{y}^-$  (Eq. 8)
16:   Calculate  $\mathcal{L}_{\text{GKR}}$  for global consistency (Eq. 9)
17:    $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{BPR}} + \alpha \mathcal{L}_{\text{PRD}} + \beta \mathcal{L}_{\text{SFD}} + \gamma \mathcal{L}_{\text{GKR}}$ 
18:   Update  $\psi \leftarrow \psi - \eta \nabla_{\psi} \mathcal{L}_{\text{total}}$  {Update only prompt}
19: end while
20: Output: Final scores  $\hat{y}_u$  using optimized  $\psi$ .
```

- **RQ1:** How does DPU compare with state-of-the-art unlearning methods in balancing *recommendation utility* and *forgetting completeness*? (Efficiency)
- **RQ2:** Is DPU computationally superior to retraining and fine-tuning baselines? (Efficiency)
- **RQ3:** How do key components contribute to the overall performance? (Ablation Study)
- **RQ4:** How sensitive is DPU to key hyperparameters? (Parameter Sensitivity)
- **RQ5:** Does DPU effectively eliminate membership signals of forgotten interactions, thereby mitigating privacy risks while preserving model behavior? (Forgetting Analysis)

## 5.1 Experiment Setup

**5.1.1 Datasets.** We evaluate DPU on three real-world benchmark datasets that are widely adopted in recommendation unlearning research [2, 38]. These datasets are selected to cover diverse graph characteristics to support a comprehensive evaluation: **ML-1M** [11]<sup>1</sup> provides dense interactions to test utility preservation against cascading errors; **Yelp** [7]<sup>2</sup> offers a large-scale sparse setting to rigorously evaluate computational efficiency; and **Netflix** [10]<sup>3</sup> serves to validate the framework’s scalability on high-volume streaming data. All datasets are randomly split into 70% training, 10% validation, and 20% testing sets. Detailed statistics are shown in Table 1.

<sup>1</sup><https://grouplens.org/datasets/movielens/1m/>

<sup>2</sup><https://business.yelp.com/data/resources/open-dataset/>

<sup>3</sup><https://www.kaggle.com/datasets/netflix-inc/netflix-prize-data/>

**Table 1: Statistics of datasets used for evaluation.**

| Dataset  | Users  | Items   | Interactions | Sparsity |
|----------|--------|---------|--------------|----------|
| ML-1M    | 6,040  | 3,706   | 1,000,209    | 95.53%   |
| Netflix  | 15,059 | 17,370  | 2,556,226    | 99.02%   |
| Yelp2018 | 15,618 | 153,151 | 1,557,462    | 99.93%   |

**5.1.2 Baselines.** To demonstrate the effectiveness of our method, we compare against representative unlearning baselines built on the LightGCN [12] backbone. We group them into two categories: **(1) Partition-based methods**, including Retrain (an upper bound with full retraining), SISA [1], and RecEraser [2], which partition the training data and retrain or aggregate affected submodels; and **(2) Fine-tuning-based methods** include GNNDelete [5] and UnlearnRec [3], which perform unlearning by fine-tuning lightweight modules or model components at unlearning time to revise representations affected by the removed interactions. **(3) Influence-based methods**, including SCIF [20] and IFRU [42], which estimate and suppress the influence of forgotten interactions without retraining. Since DPU is the first prompt-based framework for recommendation unlearning, there are no direct prompt-based baselines for comparison. All methods use identical data splits and forget/retain partitions for a fair evaluation.

**5.1.3 Evaluation Metrics.** We evaluate recommendation unlearning along three aspects, following prior work [2, 19, 38]. **(1) Model Utility.** We report Recall@K and NDCG@K with  $K \in \{10, 20\}$ ; higher is better. **(2) Unlearning Completeness.** We use Zero Retrain Forgetting (ZRF) [6], which measures the Jensen-Shannon divergence between the unlearned model and a forgetting teacher on the forget set; higher (closer to 1) indicates more complete forgetting. **(3) Efficiency.** We report total unlearning time and the number of updated parameters.

**5.1.4 Implementation Details.** All experiments are run on a single NVIDIA 4090 GPU with PyTorch. We use Adam with learning rate 0.0005, batch size 1024, embedding dimension 64, and 3 LightGCN layers. We tune prompt length  $|\psi| \in \{10, 20, \dots, 100\}$  by validation NDCG@20, and tune  $\alpha, \beta, \gamma \in \{0.1, 0.2, \dots, 1.0\}$ , using  $\alpha = 0.5$ ,  $\beta = 0.4$ , and  $\gamma = 0.5$  for all datasets. We apply early stopping with patience 10 and report averages over three runs; baselines follow official implementations unless stated otherwise.

## 5.2 Model Utility Analysis (RQ1)

We evaluate DPU’s utility on retained interactions, its forgetting completeness, and its robustness under different unlearning ratios. Results for a 1% forget ratio are reported in Table 2. Due to space limitations, we primarily visualize ML-1M and Netflix in Fig. 3–Fig. 7; Yelp exhibits consistent trends.

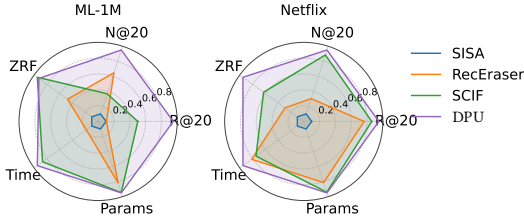
**5.2.1 Comprehensive Performance at Sparse Forget Ratios.** Under a typical sparse setting (1% forget ratio), DPU preserves utility while achieving strong forgetting:

- **Utility Preservation:** Table 2 shows that DPU matches or outperforms state-of-the-art baselines on  $\mathcal{E}_r$  in Recall@K and NDCG@K ( $K \in \{10, 20\}$ ), and remains close to the Retrain oracle. In particular, DPU narrows the gap to *Retrain* and substantially exceeds influence-based methods (e.g., IFRU, SCIF), highlighting



**Table 2: Unlearning performance on ML-1M, Netflix, and Yelp datasets. All main results are reported with a forgetting ratio of 1%. Bold denotes the best overall, underline the best baseline (excluding Retrain). We report R@K (Recall@K), N@K (NDCG@K), and the ZRF (Zero Retrain Forgetting) metric; higher is better for all metrics. Results are averaged over 3 runs; \* marks statistically significant improvement over the best baseline ( $p < 0.05$ , using paired t-test).**

| Backbone | Method     | ML-1M          |                |                |                |                | Netflix        |                |                |                |                | Yelp           |                |                |                |                |
|----------|------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
|          |            | R@10           | N@10           | R@20           | N@20           | ZRF            | R@10           | N@10           | R@20           | N@20           | ZRF            | R@10           | N@10           | R@20           | N@20           | ZRF            |
| LightGCN | Retrain    | 0.1332         | 0.3612         | 0.2084         | 0.3398         | 0.9091         | 0.2483         | 0.1837         | 0.2817         | 0.2139         | 0.8865         | 0.0359         | 0.0603         | 0.0595         | 0.0627         | 0.9167         |
|          | SISA       | 0.0347         | 0.0515         | 0.0570         | 0.0549         | 0.8724         | 0.0714         | 0.0853         | 0.1034         | 0.1217         | 0.8858         | 0.0133         | 0.0176         | 0.0237         | 0.0192         | 0.9132         |
|          | RecEraser  | 0.0343         | 0.1151         | 0.0578         | 0.2088         | 0.8921         | 0.0871         | 0.1372         | 0.1382         | 0.1565         | 0.8973         | 0.0199         | 0.0249         | 0.0317         | 0.0211         | 0.9273         |
|          | IFRU       | 0.0720         | 0.1259         | 0.1252         | 0.1341         | 0.8733         | 0.0831         | 0.1646         | 0.1432         | 0.1693         | 0.9281         | 0.0187         | 0.0263         | 0.0362         | 0.0298         | 0.9317         |
|          | SCIF       | 0.0705         | 0.1279         | 0.1192         | 0.1304         | 0.9167         | 0.0956         | 0.1655         | 0.1539         | 0.1636         | 0.9173         | 0.0195         | 0.0217         | 0.0327         | 0.0267         | 0.9245         |
|          | GNNDelete  | 0.0742         | 0.1305         | 0.1288         | 0.1390         | 0.9180         | <u>0.0985</u>  | 0.1690         | 0.1560         | <u>0.1685</u>  | 0.9210         | 0.0202         | 0.0245         | 0.0335         | 0.0278         | 0.9290         |
|          | UnlearnRec | <u>0.0751</u>  | <u>0.1320</u>  | <u>0.1305</u>  | <u>0.1415</u>  | <u>0.9250</u>  | 0.0978         | <u>0.1705</u>  | <u>0.1575</u>  | 0.1670         | <u>0.9310</u>  | <u>0.0205</u>  | <u>0.0250</u>  | <u>0.0342</u>  | <u>0.0285</u>  | <u>0.9350</u>  |
|          | DPU        | <b>0.0987*</b> | <b>0.2846*</b> | <b>0.1591*</b> | <b>0.2691*</b> | <b>0.9517*</b> | <b>0.1271*</b> | <b>0.1818</b>  | <b>0.1723*</b> | <b>0.1805</b>  | <b>0.9365</b>  | <b>0.0237*</b> | <b>0.0343*</b> | <b>0.0453*</b> | <b>0.0319</b>  | <b>0.9428*</b> |
| NGCF     | Retrain    | 0.1723         | 0.3229         | 0.2618         | 0.3064         | 0.9533         | 0.1044         | 0.1985         | 0.1705         | 0.1850         | 0.9493         | 0.0314         | 0.0347         | 0.0534         | 0.0430         | 0.9629         |
|          | SISA       | 0.0669         | 0.1213         | 0.1173         | 0.1295         | 0.8860         | 0.0373         | 0.0667         | 0.0707         | 0.0696         | 0.8934         | 0.0141         | 0.0151         | 0.0258         | 0.0199         | 0.8235         |
|          | RecEraser  | 0.1181         | 0.2086         | 0.1959         | 0.2144         | <u>0.9497</u>  | 0.0776         | 0.1402         | 0.1319         | 0.1465         | 0.9107         | 0.0244         | 0.0269         | 0.0432         | 0.0343         | 0.9159         |
|          | IFRU       | 0.1491         | 0.2649         | 0.2305         | 0.2643         | 0.9320         | 0.0800         | 0.1424         | 0.1338         | 0.1501         | 0.9109         | 0.0119         | 0.0119         | 0.0239         | 0.0170         | 0.9106         |
|          | SCIF       | 0.1479         | 0.2450         | 0.2255         | 0.2464         | 0.8856         | 0.0998         | 0.1836         | 0.1626         | 0.1754         | 0.7648         | 0.0278         | 0.0315         | 0.0463         | 0.0382         | 0.9192         |
|          | GNNDelete  | 0.1515         | 0.2680         | 0.2355         | 0.2690         | 0.9410         | 0.1005         | 0.1850         | 0.1640         | 0.1765         | 0.9150         | 0.0285         | 0.0325         | 0.0485         | 0.0395         | 0.9220         |
|          | UnlearnRec | 0.1508         | 0.2665         | 0.2340         | 0.2675         | 0.9450         | 0.1002         | <u>0.1865</u>  | <u>0.1655</u>  | <u>0.1770</u>  | <u>0.9250</u>  | <u>0.0290</u>  | <u>0.0330</u>  | <u>0.0490</u>  | <u>0.0400</u>  | <u>0.9280</u>  |
|          | DPU        | <b>0.1958*</b> | <b>0.3354*</b> | <b>0.2890*</b> | <b>0.3321*</b> | <b>0.9643*</b> | <b>0.1042*</b> | <b>0.1986*</b> | <b>0.1701*</b> | <b>0.1849*</b> | <b>0.9516*</b> | <b>0.0350*</b> | <b>0.0391*</b> | <b>0.0582*</b> | <b>0.0478*</b> | <b>0.9408*</b> |



**Figure 3: Radar comparison on ML-1M and Netflix (LightGCN) across five metrics. IFRU is omitted for clarity as SCIF represents influence-based methods. DPU demonstrates a superior trade-off between unlearning effectiveness, utility, and efficiency.**

the benefit of position-aware ranking distillation in preserving the memorizing teacher’s high-quality rankings.

- **Forgetting Completeness:** DPU achieves the highest ZRF scores (e.g., 0.9517 on ML-1M), exceeding the strongest baseline (SCIF) by 3.5%. This indicates more complete removal of targeted interaction influence, approaching the behavior of a model never trained on the forget set.

**5.2.2 Holistic Trade-off Analysis.** We use radar charts (Figure 3) to assess the Pareto trade-off of DPU across five dimensions: utility (R@20, N@20), completeness (ZRF), and efficiency (Time, Params). DPU exhibits a larger and more balanced performance profile than competing methods. Compared with partition-based methods that often compromise global utility for modular retraining, and influence-based methods that are limited by approximation error, DPU delivers strong results across all objectives. Its lightweight, prompt-guided design updates only a small number of parameters

and converges quickly, while achieving state-of-the-art utility and forgetting performance. Overall, DPU mitigates the unlearning–utility trade-off that commonly arises in existing frameworks.

**5.2.3 Robustness to Increasing Forget Ratios.** Figure 4 evaluates DPU from 1% to 10% forget ratios, demonstrating resilient performance amid growing deletion demands.

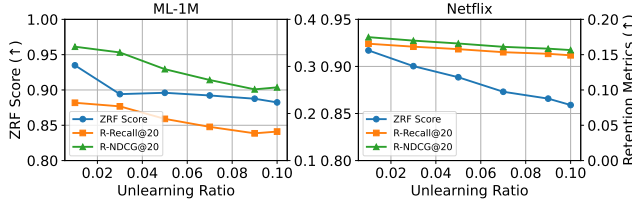
- **Forgetting Robustness:** ZRF remains high with minimal decay—e.g., subtle drops on ML-1M and Netflix—as prompts scalably adapt to larger  $\mathcal{E}_f$  via structure-aware distillation, maintaining erasure efficacy without proportional computational escalation.
- **Utility Preservation Robustness:** Retained metrics exhibit controlled degradation: on ML-1M, R@20 and N@20 fall by 27.47% and 25.2%; on Netflix, by 9.91% and 10.47%. This resilience arises from memorizing teacher regularization, preserving global ranking stability even as unlearning perturbs more graph topology, outperforming baselines prone to amplified utility loss.

Overall, these findings validate DPU’s design, enabling efficient, verifiable unlearning in GNN recommenders without compromising core performance.

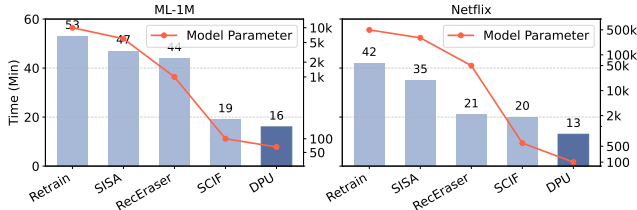
### 5.3 Efficiency Analysis (RQ2)

We then evaluate the computational efficiency of DPU during unlearning and examine how prompt length affects both unlearning quality and retained utility. All experiments are conducted on an NVIDIA 4090 GPU.

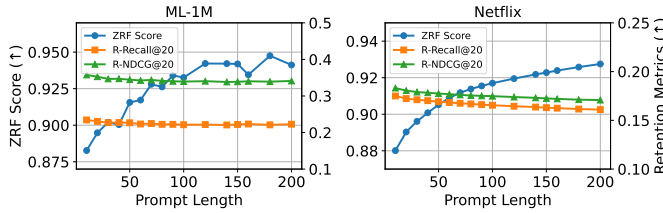
**5.3.1 Computational Efficiency.** DPU offers clear efficiency advantages over retraining-based methods in both unlearning time and the number of updated parameters.



**Figure 4: Robustness analysis under varying unlearning ratios (0.01–0.1) on ML-1M and Netflix datasets, built upon a LightGCN backbone.**



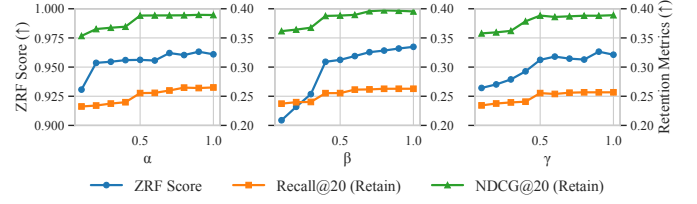
**Figure 5: Computational efficiency comparison on ML-1M and Netflix datasets, using LightGCN as the backbone model. Bars represent unlearning time in minutes (lower is better), and lines indicate the number of trainable parameters excluding the embedding layer (lower is better).**



**Figure 6: Unlearning and retention performance under varying prompt lengths on the ML-1M and Netflix datasets, using LightGCN as the backbone model.**

- **Unlearning Time:** As shown in Figure 5, DPU consistently achieves the shortest unlearning time, delivering a 3–4× speedup over retraining-based baselines. This corresponds to a **66.7%–75% reduction** in total unlearning time, making DPU practical for dynamic, real-world settings.
- **Inference Efficiency:** At inference, DPU matches the runtime of the original model: the learned prompt is added to input features without introducing additional computation, so unlearning does not slow down recommendation serving.

**5.3.2 Prompt Length Efficiency.** Beyond time, DPU updates orders of magnitude fewer parameters. We study how the prompt capacity (number of prompt basis vectors  $K_p$ ) affects forgetting and retention by varying  $K_p$  from 10 to 200. Figure 8 summarizes the results. Increasing  $K_p$  generally improves ZRF, indicating more complete forgetting. On ML-1M, ZRF rises from 0.8827 ( $K_p = 10$ ) to 0.9475 ( $K_p = 200$ ), a 7.34% improvement. In contrast, Recall@20



**Figure 7: Sensitivity analysis of the loss weights  $\alpha$ ,  $\beta$ , and  $\gamma$  on the ML-1M dataset, using LightGCN as the backbone model. Each plot shows the effect of varying a single weight on ZRF, Recall@20, and NDCG@20.**

**Table 3: Ablation results on the ML-1M dataset base on LightGCN showing the impact of removing each component (PRD, SFD, GKR) on forgetting and retention performance (Recall@20) across different top-K values.**

| Variant    | Set                 | Top-K  |        |        |        |        |
|------------|---------------------|--------|--------|--------|--------|--------|
|            |                     | 5      | 10     | 20     | 30     | 50     |
| Full Model | Retain $\uparrow$   | 0.0916 | 0.1498 | 0.2314 | 0.2924 | 0.3803 |
|            | Forget $\downarrow$ | 0.0274 | 0.0359 | 0.0584 | 0.0809 | 0.1092 |
| w/o PRD    | Retain $\uparrow$   | 0.0609 | 0.1019 | 0.1694 | 0.2203 | 0.2984 |
|            | Forget $\downarrow$ | 0.0106 | 0.0271 | 0.0476 | 0.0657 | 0.0922 |
| w/o SFD    | Retain $\uparrow$   | 0.0906 | 0.1487 | 0.2295 | 0.2917 | 0.3817 |
|            | Forget $\downarrow$ | 0.0585 | 0.0810 | 0.1278 | 0.1617 | 0.1994 |
| w/o GKR    | Retain $\uparrow$   | 0.0943 | 0.1513 | 0.2341 | 0.2947 | 0.3827 |
|            | Forget $\downarrow$ | 0.0309 | 0.0522 | 0.0771 | 0.1017 | 0.1404 |

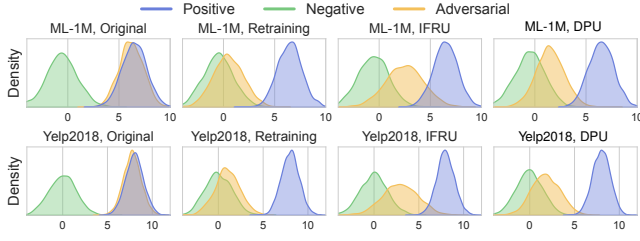
and NDCG@20 vary only slightly, indicating robust utility preservation. Overall, DPU benefits from increased prompt capacity while maintaining stable performance on retained interactions.

## 5.4 Ablation Study (RQ3)

To measure how much each distillation signal contributes, we run an ablation study. We compare the full DPU with three variants that each remove one component: **w/o PRD**, **w/o SFD**, and **w/o GKR**. PRD preserves top-K ranking quality on retained data, SFD enforces effective forgetting of target interactions, and GKR maintains global ranking stability. Results on ML-1M in Table 3 yield the following observations:

- **Impact of PRD on Utility Preservation:** Removing the PRD leads to the most severe degradation in retained set performance (e.g., Recall@10 drops from 0.1498 to 0.1019). This confirms that PRD is the primary "utility engine," anchoring the student prompt to the memorizing teacher's high-order ranking manifold and ensuring that top-K recommendations remain unaffected by the unlearning process.
- **Impact of SFD on Forgetting Efficacy:** The exclusion of SFD causes a significant surge in Recall@10 on the forget set (from 0.0359 to 0.0810). This indicates a failure to neutralize the targeted interaction influence. SFD acts as the "forgetting inducer," effectively suppressing the weights of sensitive edges by aligning the student with an "ignorant" forgetting teacher.





**Figure 8: Visualization for the predictions of positive, negative, and adversarial edges in the ML-1M and Yelp2018 datasets, using LightGCN as the backbone model.**

- **Impact of GKR on Structural Stability:** While removing the GKR has a marginal effect on raw utility, it leads to a noticeable decline in unlearning completeness (ZRF). This suggests that GKR serves as a global stabilizer; by penalizing pairwise order inversions across the entire item space, it prevents the prompt tuning process from over-fitting to local deletions and maintains the global ranking consistency.

The full model achieves the optimal Pareto front, demonstrating that the three strategies are complementary.

## 5.5 Parameter Sensitivity Analysis (RQ4)

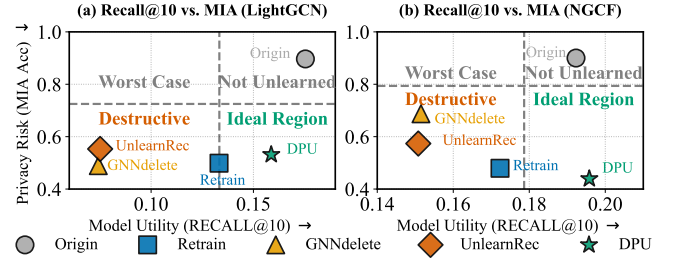
To answer RQ4, we analyze the impact of key hyperparameters on the model’s performance. Specifically, we study the sensitivity of DPU to the loss weights  $\alpha$  (PRD),  $\beta$  (SFD), and  $\gamma$  (GKR).

As shown in Figure 7 on ML-1M, all three weights exhibit a similar trend: unlearning completeness (ZRF) and retained utility (Recall@20, NDCG@20) improve as the weight increases from 0.1 to 0.4, and then largely stabilize beyond 0.5 with minimal variance. For example, NDCG@20 converges to  $\approx 0.3950$  for  $\alpha \geq 0.4$ , and we observe comparable saturation behavior for  $\beta$  and  $\gamma$ . Overall, these results suggest that each distillation term should be sufficiently weighted, while the objective remains robust to the exact choice once the weights exceed a moderate threshold.

## 5.6 Privacy and Forgetting Verification(RQ5)

To answer RQ5, we evaluate whether DPU eliminates membership signals of forgotten interactions without altering model behavior. We conduct this analysis from both a distributional perspective and an adversarial privacy perspective.

**5.6.1 Distributional Shift Analysis.** To visually verify if the model truly “forgets,” we analyze the shift in prediction score distributions before and after unlearning. Following standard unlearning protocols [3, 32, 33], we first train a GNN on the original data and then construct adversarial edges by sampling user–item pairs that the trained model considers least likely to occur. This forces the model to memorize incorrect preferences. The goal of unlearning is to remove the influence of these injected edges so that the model again assigns them low confidence, similar to unseen *Negative* interactions. Figure 8 presents the Kernel Density Estimation (KDE) of the prediction scores for three sets on ML-1M and Yelp2018: *Positive* (retained), *Negative* (unseen), and *Adversarial* (the forget set  $\mathcal{E}_f$ ). In the Original model, the score distribution of *Adversarial* edges largely overlaps with that of *Positive* interactions, indicating that these



**Figure 9: Privacy–utility trade-off under membership inference attacks on LightGCN and NGCF. The bottom-right region indicates high utility with low privacy leakage.**

unlikely edges are strongly memorized after training. Conversely, the **Retrain** approach (our gold standard) shifts the *Adversarial* curve to align almost perfectly with the *Negative* distribution, effectively “forgetting” their existence. The influence-based baseline **IFRU** partially reduces this overlap, but its *Adversarial* scores remain scattered between *Positive* and *Negative*, DPU closely follows the retrain behavior across both datasets, consistently pushing the adversarial score distribution toward that of *Negative* interactions. This result shows that our prompt-guided distillation effectively removes the membership signals of forgotten interactions, making them statistically indistinguishable from truly unseen data.

**5.6.2 Privacy–Utility Trade-off under Membership Inference Attacks.** While distributional alignment suggests successful forgetting, we further conduct Membership Inference Attacks (MIA) and analyze the Privacy Utility Trade-off. We map the results onto a landscape where the x-axis represents Model Utility (Recall@10, higher is better) and the y-axis represents Privacy Risk (MIA Accuracy, where 0.5 indicates ideal random guessing). Consequently, the optimal performance lies in the bottom-right “Ideal Region”. As visualized in Figure 9, the Original model exhibits high MIA accuracy, reflecting substantial privacy leakage. Conversely, DPU consistently lands in the green-boxed “Ideal Region” (bottom-right). It achieves a privacy risk profile ( $\approx 0.53$ ) comparable to the Retrain oracle while avoiding the utility degradation seen in baselines. These results confirm that the distributional alignment observed earlier translates into robust defense, establishing DPU on the optimal Pareto frontier between privacy and utility.

## 6 Conclusion

In this work, we present DPU, which redefines recommendation unlearning as a lightweight, prompt-guided adaptation task rather than a parameter re-optimization task. By freezing the GNN backbone, DPU enables reversible, structural unlearning through attention-based prompts that “absorb” the influence of deleted edges directly in the embedding space. The dual-teacher distillation strategy further balances forgetting and retention by stabilizing ranking performance on retained data while suppressing membership signals of forgotten interactions. Extensive experiments demonstrate that DPU achieves unlearning behavior close to retraining with substantially lower computational cost, making it a practical and efficient solution for real-world recommendation unlearning.

## References

- [1] Lucas Bourtole, Varun Chandrasekaran, Christopher A Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. 2021. Machine unlearning. In *2021 IEEE Symposium on security and privacy (SP)*. IEEE, 141–159.
- [2] Chong Chen, Fei Sun, Min Zhang, and Bolin Ding. 2022. Recommendation unlearning. In *Proceedings of the ACM Web Conference 2022*. 2768–2777.
- [3] Guoxuan Chen, Lianghao Xia, and Chao Huang. 2025. Pre-training for Recommendation Unlearning. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2112–2121.
- [4] Min Chen, Zhikun Zhang, Tianhao Wang, Michael Backes, Mathias Humbert, and Yang Zhang. 2022. Graph unlearning. In *2022 ACM SIGSAC*. 499–513.
- [5] Jiali Cheng, George Dasoulas, Huan He, Chirag Agarwal, and Marinka Zitnik. 2023. GNNDelete: A General Strategy for Unlearning in Graph Neural Networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [6] Vikram S. Chundawat, Ayush K. Tarun, Murari Mandal, and Mohan Kankanhalli. 2023. Can Bad Teaching Induce Forgetting? Unlearning in Deep Networks Using an Incompetent Teacher. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 37. 7210–7217.
- [7] Yan Cui. 2015. An evaluation of yelp dataset. *arXiv preprint arXiv:1512.06915* (2015).
- [8] Le Dai, Yu Yin, Enhong Chen, and Hui Xiong. 2024. Unifying Graph Retrieval and Prompt Tuning for Graph-Grounded Text Classification. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2682–2686.
- [9] Taoran Fang, Yunchao Zhang, Yang Yang, Chunping Wang, and Lei Chen. 2023. Universal prompt tuning for graph neural networks. *Advances in Neural Information Processing Systems* 36 (2023), 52464–52489.
- [10] Andrey Feuerverger, Yu He, and Shashi Khatri. 2012. Statistical significance of the Netflix challenge. *Statist. Sci.* 27, 2 (2012), 202–231.
- [11] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems (TIIS)* 5, 4 (2015), 1–19.
- [12] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 639–648.
- [13] Yifan Hu, Yehuda Koren, and Chris Volinsky. 2008. Collaborative Filtering for Implicit Feedback Datasets. In *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*. IEEE, 263–272.
- [14] Hai Huang, Jiaming Mu, Neil Zhenqiang Gong, Qi Li, Bin Liu, and Mingwei Xu. 2021. Data poisoning attacks to deep learning based recommender systems. *arXiv preprint arXiv:2101.02644* (2021).
- [15] Uday Kamath, Kevin Keenan, Garrett Somers, and Sarah Sorenson. 2024. Prompt-based Learning. In *Large Language Models: A Deep Dive: Bridging Theory and Practice*. Springer, 83–133.
- [16] Maurice G Kendall. 1938. A new measure of rank correlation. *Biometrika* 30, 1-2 (1938), 81–93.
- [17] Hyunjun Kim, Sangyong Lee, and Simon S Woo. 2024. Layer attack unlearning: Fast and accurate machine unlearning via layer level attack and knowledge distillation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 21241–21248.
- [18] Yuyuan Li, Chaochao Chen, Yizhao Zhang, Weiming Liu, Lingjuan Lyu, Xiaolin Zheng, Dan Meng, and Jun Wang. 2023. Ultrare: Enhancing receraser for recommendation unlearning via error decomposition. *Advances in Neural Information Processing Systems* 36 (2023), 12611–12625.
- [19] Yuyuan Li, Chaochao Chen, Xiaolin Zheng, Junlin Liu, and Jun Wang. 2024. Making recommender systems forget: Learning and unlearning for erasable recommendation. *Knowledge-Based Systems* 283 (2024), 111124.
- [20] Yuyuan Li, Chaochao Chen, Xiaolin Zheng, Yizhao Zhang, Biao Gong, Jun Wang, and Linxun Chen. 2023. Selective and Collaborative Influence Function for Efficient Recommendation Unlearning. *Expert Systems with Applications* 234 (2023), 121025.
- [21] Yuyuan Li, Xiaohua Feng, Chaochao Chen, and Qiang Yang. 2024. A Survey on Recommendation Unlearning: Fundamentals, Taxonomy, Evaluation, and Open Questions. *arXiv:2412.12836* (2024).
- [22] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *Comput. Surveys* 55, 9 (2023), 1–35.
- [23] Wenyan Liu, Juncheng Wan, Xiaoling Wang, Weinan Zhang, Dell Zhang, and Hang Li. 2022. Forgetting fast in recommender systems. *arXiv preprint arXiv:2208.06875* (2022).
- [24] Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. 2022. P-tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 61–68.
- [25] Roger Penrose. 1955. A generalized inverse for matrices. In *Mathematical proceedings of the Cambridge philosophical society*, Vol. 51. Cambridge University Press, 406–413.
- [26] Zhen Qin, Rolf Jagerman, Rama Kumar Pasumarthi, Honglei Zhuang, He Zhang, Aijun Bai, Kai Hui, Le Yan, and Xuanhui Wang. 2023. Rd-suite: A benchmark for ranking distillation. *Advances in Neural Information Processing Systems* 36 (2023), 35748–35760.
- [27] Yash Sinha, Murari Mandal, and Mohan Kankanhalli. 2025. Distill to Delete: Unlearning in Graph Networks With Knowledge Distillation. *IEEE Transactions on Neural Networks and Learning Systems (TNNLS)* (2025).
- [28] Xiangguo Sun, Jiawen Zhang, Xixi Wu, Hong Cheng, Yun Xiong, and Jia Li. 2023. Graph prompt learning: A comprehensive survey and beyond. *arXiv preprint arXiv:2311.16534* (2023).
- [29] Paul Voigt and Axel von dem Bussche. 2017. *The EU General Data Protection Regulation (GDPR): A Practical Guide* (1st ed., cham: springer international publishing ed.). Cham.
- [30] Bichen Wang, Yuzhe Zi, Yixin Sun, Yanyan Zhao, and Bing Qin. 2024. Rkld: Reverse kl-divergence-based knowledge distillation for unlearning personal information in large language models. *arXiv preprint arXiv:2406.01983* (2024).
- [31] Yuhang Wang, Zhenxing Niu, Haoxuan Ji, Guangyu He, Haichang Gao, and Gang Hua. 2025. MLLM Machine Unlearning via Visual Knowledge Distillation. *arXiv preprint arXiv:2512.11325* (2025).
- [32] Jiancan Wu, Yi Yang, Yuchun Qian, Yongduo Sui, Xiang Wang, and Xiangnan He. 2023. Gif: A general graph unlearning strategy via influence function. In *Proceedings of the ACM Web Conference 2023*. 651–661.
- [33] Kun Wu, Jie Shen, Yue Ning, Ting Wang, and Wendy Hui Wang. 2023. Certified edge unlearning for graph neural networks. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2606–2617.
- [34] Xin Xin, Liu Yang, Ziqi Zhao, Pengjie Ren, Zhumin Chen, Jun Ma, and Zhaochun Ren. 2024. On the effectiveness of unlearning in session-based recommendation. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining (WSDM)*. 855–863.
- [35] Mimeo Xu, Jiankai Sun, Xin Yang, Kevin Yao, and Chong Wang. 2023. Netflix and forget: Efficient and exact machine unlearning from bi-linear recommendations. *arXiv preprint arXiv:2302.06676* (2023).
- [36] Yuhao Yang, Lianghao Xia, Da Luo, Kangyi Lin, and Chao Huang. 2024. GraphPro: Graph Pre-training and Prompt Learning for Recommendation. In *Proceedings of the ACM Web Conference (WWW)*. 2035–2046.
- [37] Shanshan Ye and Jie Lu. 2023. Sequence Unlearning for Sequential Recommender Systems. In *Proceedings of the Australasian Joint Conference on Artificial Intelligence (AJCAI)*. 403–415.
- [38] Xiaoyu You, Jianwei Xu, Mi Zhang, Zechen Gao, and Min Yang. 2024. Rrl: Recommendation reverse learning. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 9296–9304.
- [39] Yawen Zeng. 2022. Point prompt tuning for temporally language grounding. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*. 2003–2007.
- [40] Peiyan Zhang, Yuchen Yan, Xi Zhang, Liying Kang, Chaozhao Li, Feiran Huang, Senzhang Wang, and Sunghun Kim. 2024. Gpt4rec: Graph prompt tuning for streaming recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1774–1784.
- [41] Qianru Zhang, Lianghao Xia, Xuheng Cai, Siu-Ming Yiu, Chao Huang, and Christian S Jensen. 2024. Graph augmentation for recommendation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 557–569.
- [42] Yang Zhang, Zhiyu Hu, Yimeng Bai, Fuli Feng, Jiancan Wu, Qifan Wang, and Xiangnan He. 2024. Recommendation Unlearning via Influence Function. *ACM Transactions on Recommender Systems (TORS)* 3, 2 (2024), 1–23.
- [43] Wenyue Zheng, Ximeng Liu, Yuyang Wang, and Xuanwei Lin. 2023. Graph unlearning using knowledge distillation. In *International Conference on Information and Communications Security*. Springer, 485–501.
- [44] Yu Zhou, Dian Zheng, Qijie Mo, Renjie Lu, Kun-Yu Lin, and Wei-Shi Zheng. 2025. Decoupled distillation to erase: A general unlearning method for any class-centric tasks. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 20350–20359.